

Afrobotics BIT — Brand Insertion Technology

Application Presentation & Architecture Overview

Document Version: 1.0 **Date:** 22 July 2026 **Status:** Engine Verification Ready

Table of Contents

- 1. Executive Summary
- 2. Infrastructure & Technology Stack
- 3. System Architecture
- 4. Feature Catalogue
- 5. User Roles & Permissions
- 6. Application Flow
- 7. Content Pipeline
- 8. API Surface
- 9. Security Architecture
- 10. Database Schema

1. Executive Summary

Afrobotics Brand Insertion Technology (BIT) is an AI-powered video inventory creation platform that enables brands to be inserted into existing video content after production. It transforms suitable surfaces inside video — billboards, screens, walls, signage, product spaces — into monetisable advertising inventory while appearing visually indistinguishable from the original scene.

Vision

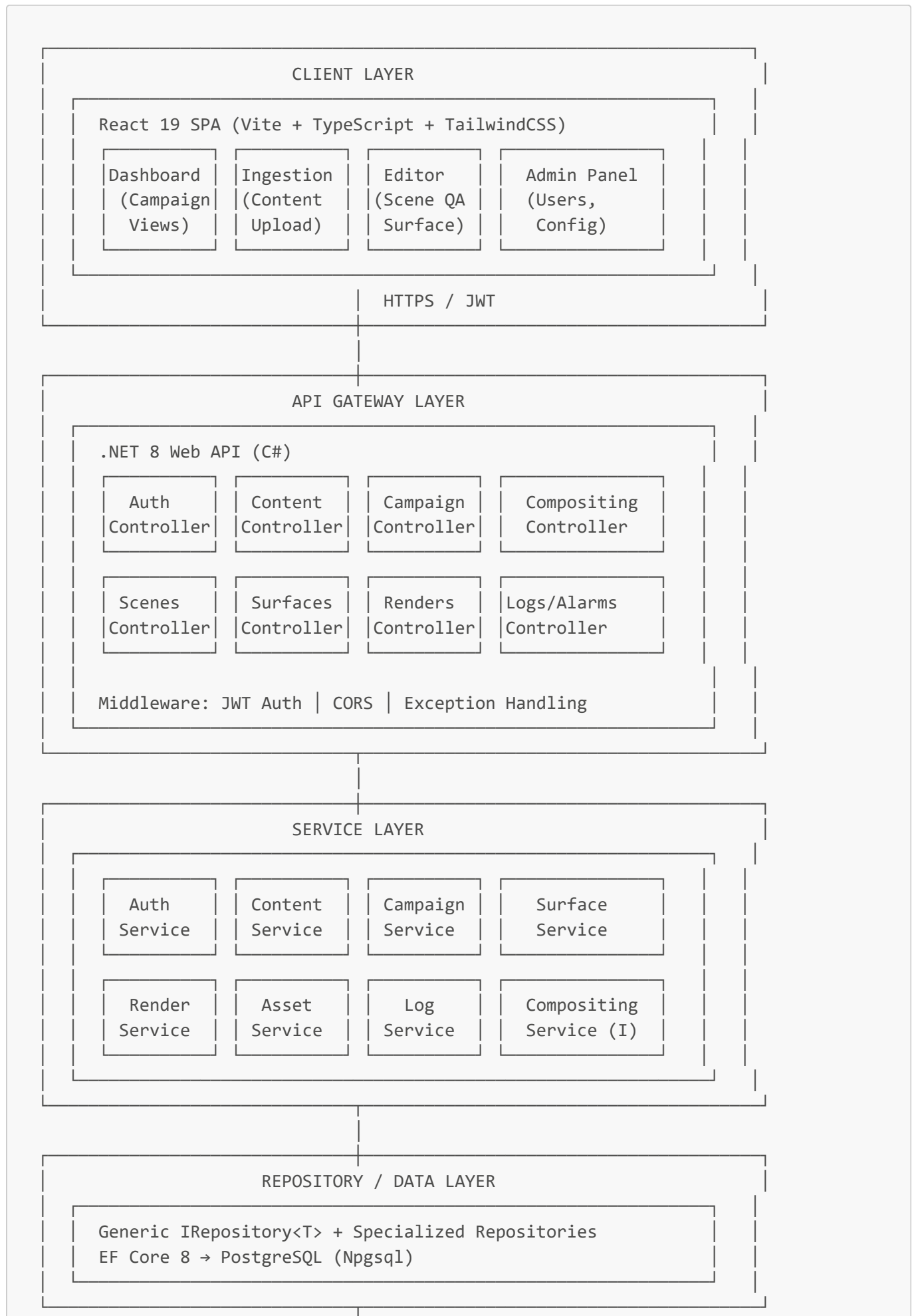
"Turn every surface in existing premium video content into a monetisable, brand-safe advertising opportunity — without disrupting the viewer experience."

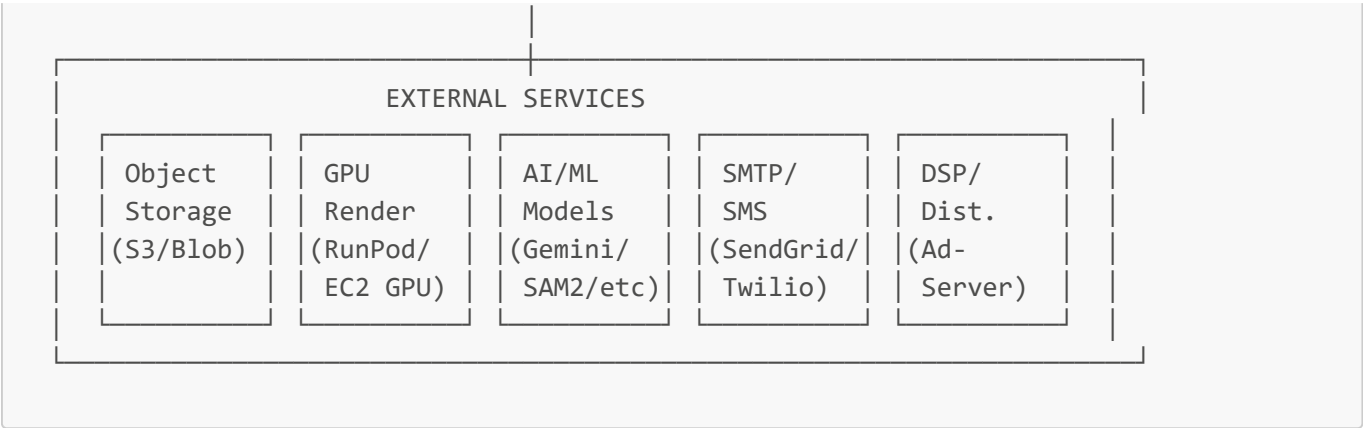
Key Capabilities

- **Content Ingestion** — Accept video, transcode, detect scenes, extract metadata
- **AI Scene Analysis** — Computer-vision surface detection, depth estimation, orientation mapping
- **Placement Recommendation** — Score and rank candidate surfaces; brand-safety enforcement
- **Motion Tracking** — Lock brand assets to surfaces through camera movement
- **AI Compositing** — Perspective warp, relight, blur, shadow matching for photo-realism
- **GPU Rendering** — Batch render broadcast/streaming/social output
- **Campaign Management** — Advertiser campaigns, asset library, scheduling, regional targeting
- **Approval Workflow** — Mandatory human-in-the-loop approval; audit trail
- **Analytics & BI** — Exposure metrics, performance reporting, real-time dashboards
- **Enterprise Operations** — RBAC, JWT auth, event logging, alarms, redundancy

2. Infrastructure & Technology Stack

2.1 Infrastructure Diagram





2.2 Technology Stack

Layer	Technology	Version
Frontend Framework	React (with Hooks, Context)	19.0
Build Tool	Vite	6.2
Language	TypeScript	5.8
CSS Framework	TailwindCSS	4.1
Routing	React Router DOM	7.18
Animation	Motion (Framer Motion)	12.23
Icons	Lucide React	0.546
Backend Runtime	.NET (ASP.NET Core)	8.0
Language	C# 12	—
ORM	Entity Framework Core	8.0
Database	PostgreSQL (via Npgsql)	16+
Authentication	JWT Bearer	8.0
Background Jobs	Hangfire	1.8
AI SDK (Client)	Google GenAI (@google/genai)	2.4
Password Hashing	BCrypt.Net-Next	4.2

2.3 Planned / Swappable Services

Layer	Candidate Technologies	Status
Scene Detection	FFmpeg + custom scene-cut library	Build
Segmentation / Object Detection	SAM 2, YOLOv8, Detectron2	TBC (Wrappable)
Motion Tracking	OpenCV planar tracker + point-tracking fallback	TBC (Wrappable)

Layer	Candidate Technologies	Status
Compositing Engine	Runway Gen-4 API, SAM 2 + IC-Light, proprietary homography solver	TBC (Swappable via <code>ICompositingService</code>)
GPU Rendering	RunPod, AWS EC2 GPU (G5 instances), FFmpeg, Blender	TBC
Object Storage	AWS S3, Azure Blob Storage, MinIO (on-prem)	TBC
Notifications	SendGrid/Mailgun (SMTP), Twilio/Africa's Talking (SMS)	TBC
Message Queue	RabbitMQ, Azure Service Bus, AWS SQS	TBC
Containerization	Docker, Kubernetes (AKS/EKS)	Planned

3. System Architecture

3.1 Architectural Patterns

Pattern	Usage
Layered Architecture	Controllers → Services → Repositories → Database
Repository Pattern	Generic <code>IRepository<T></code> with specialized interfaces per aggregate
Strategy Pattern	<code>ICompositingService</code> , <code>ISurfaceDetectionService</code> , <code>IBrandAnalysisService</code> allow swapping AI engines without re-architecting
Dependency Injection	All services registered in DI container (<code>Program.cs</code>)
JWT Token Auth	Stateless authentication via Bearer tokens
SPA with API Proxy	Vite dev proxy <code>/api</code> → .NET backend; same origin in production

3.2 Engine Swappability

The platform supports runtime-configurable AI engines via `IPlatformSettingsService`:

Capability	Interface	Available Engines
Surface Detection	<code>ISurfaceDetectionService</code>	<code>basic</code> (default), <code>replicate</code> (Replicate API), <code>google</code> (Google Vision)
Brand Analysis	<code>IBrandAnalysisService</code>	<code>basic</code> (default), <code>google</code> (Google Vision), <code>gemini</code> (Gemini)
Compositing	<code>ICompositingService</code>	<code>basic</code> (default), <code>opencv</code> (OpenCV)

Admin changes engine settings via the Admin Console → Settings → Engine tab. Changes take effect on next app restart.

3.3 Middleware Pipeline

```
HTTP Request
→ CORS (AllowFrontendClient)
→ Authentication (JWT Bearer)
→ Authorization
→ UsageTrackingMiddleware (MReq 22)
→ Controller
→ ExceptionHandlingMiddleware (production only)
→ HTTP Response
```

4. Feature Catalogue

4.1 Frontend Features

Module	Component	Description
Authentication	Login Gate (<code>App.tsx</code>)	Secure sign-in with pre-configured role cards, forgot password, token refresh, idle timeout (28min + 60s countdown)
Theme System	Theme Toggle	Dark/Light mode switch, persisted to <code>localStorage</code> , full Tailwind dark mode coverage (cards, inputs, tables, text, borders)
Campaign Workspace	<code>CampaignDashboard</code>	Per-campaign pipeline progress, budget/region/asset/render stats, quick-action cards
Campaign Sidebar	<code>CampaignSidebar</code>	URL-driven navigation (dashboard, assets, content, placements, renders, reports, admin, telemetry, analytics)
Campaign Selector	<code>CampaignSelector</code>	Dropdown campaign switcher with asset counts, direct navigation
Asset Management	<code>CampaignsTab</code>	Create/edit/delete creative assets, file upload, brand category assignment, campaign association/unassociation
Content Ingestion	<code>IngestionTab</code>	Video upload with chunked upload (>100MB), progress bar, pipeline stage indicators, delete content, AI scene detection trigger
Pipeline Controls	<code>IngestionTab</code>	Re-transcode, Re-detect Scenes, Reset Pipeline buttons with loading states and error messages

Module	Component	Description
Pipeline Progress	<code>PipelineProgress</code>	Visual 4-step progress indicator (Staging→Transcoding→SceneDetection→Completed)
QA Workbench	<code>EditorTab</code>	Scene/surface browsing, surface approval/rejection with reason, asset placement on surfaces, AI asset suggestions, compositing preview
Scene Approval	<code>EditorTab</code>	Scene-level approval workflow, AI-generated scene metadata display
GPU Compositing	<code>ComposerTab</code>	Campaign→Asset→Preset selection, render queue with status, stitching console simulator
Telemetry	<code>TelemetryTab</code>	Paginated event logs with severity/search filters, paginated alarms with active/resolved toggle, alarm simulation, alarm clearing
Analytics	<code>AnalyticsTab</code>	8 summary cards (Total Content, Scenes Indexed, Ad Surfaces, Renders, Active Campaigns, Active Alarms, Content 7d, Renders 7d), average render time
Admin Console	<code>AdminConsoleTab</code>	User CRUD (create/edit/suspend/delete), platform settings (SMTP/Upload/Pipeline/Engine), brand-safety exclusion list, role requests panel
Brand Safety	<code>BrandSafetyPanel</code>	Permanent exclusion list — add-only, toggle active/inactive, category + description
Role Requests	<code>RoleRequestsPanel</code>	Approve/reject role elevation requests with filter (All/Pending/Approved/Rejected)
Settings	<code>SettingsPanel</code>	SMTP config, upload limits, pipeline thresholds, AI engine selection, test email button
Notifications	<code>AttentionBell</code>	Real-time attention message notifications
Notification Prefs	<code>NotificationPreferencesPanel</code>	Per-user mute toggles for notification types (RenderCompleted, CampaignCreated, etc.)

4.2 Backend Features

Module	Controllers	Description
Auth	<code>AuthController</code>	Login, token refresh, forgot password, change password
Users	<code>UsersController</code>	CRUD, role management, account status
User Profile	<code>UserProfileController</code>	Current user profile, notification preferences
Role Requests	<code>AdminRoleRequestsController</code>	List, approve, reject role elevation requests

Module	Controllers	Description
Content	<code>ContentController</code>	Upload (chunked + direct), list, delete, scene detection trigger, pipeline stage transitions
Scenes	<code>ScenesController</code>	List scenes per content, update QA status, AI scene analysis
Surfaces	<code>SurfacesController</code>	List surfaces per scene, approve/reject with audit trail
Campaigns	<code>CampaignsController</code>	CRUD, naming code validation
Assets	<code>AssetsController</code>	CRUD, file upload, campaign association/unassociation
Renders	<code>RendersController</code>	Queue render jobs, list renders, status tracking
Compositing	<code>CompositingController</code>	Preview composite frame generation
Approvals	<code>ApprovalsController</code>	Approval audit trail
Alarms	<code>AlarmsController</code>	List, trigger, clear alarms
Logs	<code>LogsController</code>	Event log retrieval with pagination and filters
Notifications	<code>NotificationsController</code>	Send and manage notifications
Stats	<code>StatsController</code>	Platform summary statistics
Usage	<code>UsageController</code>	Usage record tracking and archiving
Brand Safety	<code>BrandSafetyController</code>	Add/toggle brand safety exclusion rules
Admin Settings	<code>AdminSettingsController</code>	Platform settings CRUD, test email endpoint
Attention	<code>AttentionController</code>	Attention message management

4.3 Background Jobs (Hangfire)

Job	Schedule	Description
Cleanup Chunk Temp	Daily	Removes orphaned chunked upload temp directories
Archive Usage Records	Weekly	Archives old usage tracking records

5. User Roles & Permissions

5.1 Role Definitions

Role	Access Level	Capabilities
Admin	Full Platform	User management, platform settings, brand-safety rules, role request review, all campaign views, content ingestion, QA workbench, render queue, telemetry, analytics
Editor	Operational	Campaign workspace, content ingestion, scene QA, surface approval, asset management, render queue, telemetry, analytics. No user management or platform settings.
Advertiser	Campaign-Focused	Campaign dashboard, asset upload, placement review, render status, reports. Read-only on content & telemetry.

5.2 Pre-Seeded Users

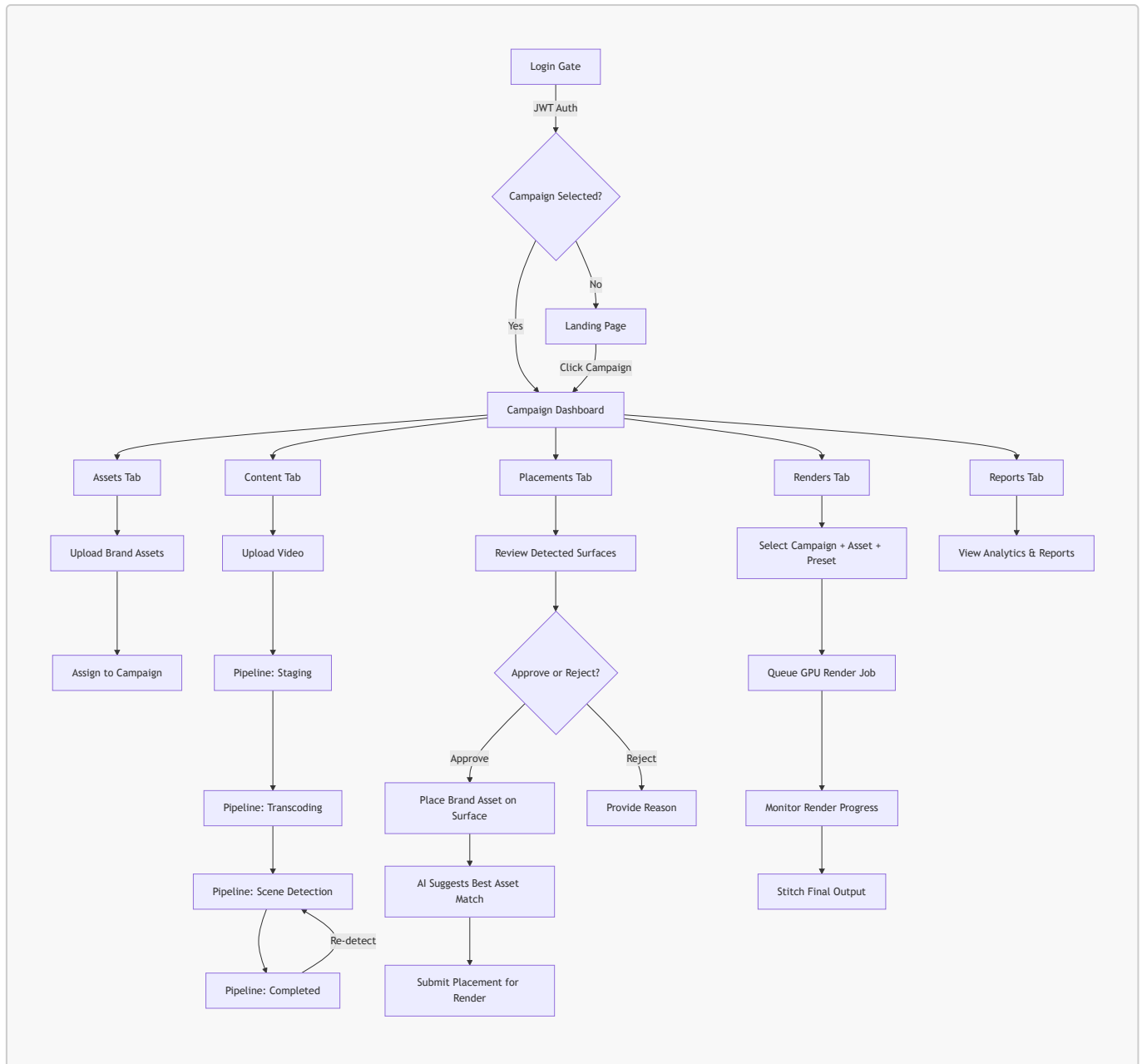
Name	Email	Password	Role
Sabelo Nkosi	<code>admin@afrobotics.co.za</code>	<code>admin123</code>	Admin
Sfiso Dlamini	<code>loverboy.sfiso@gmail.com</code>	<code>editor123</code>	Editor
Thabo Ndlovu	<code>advertiser@afrobotics.co.za</code>	<code>advertiser123</code>	Advertiser

5.3 Role Request Workflow

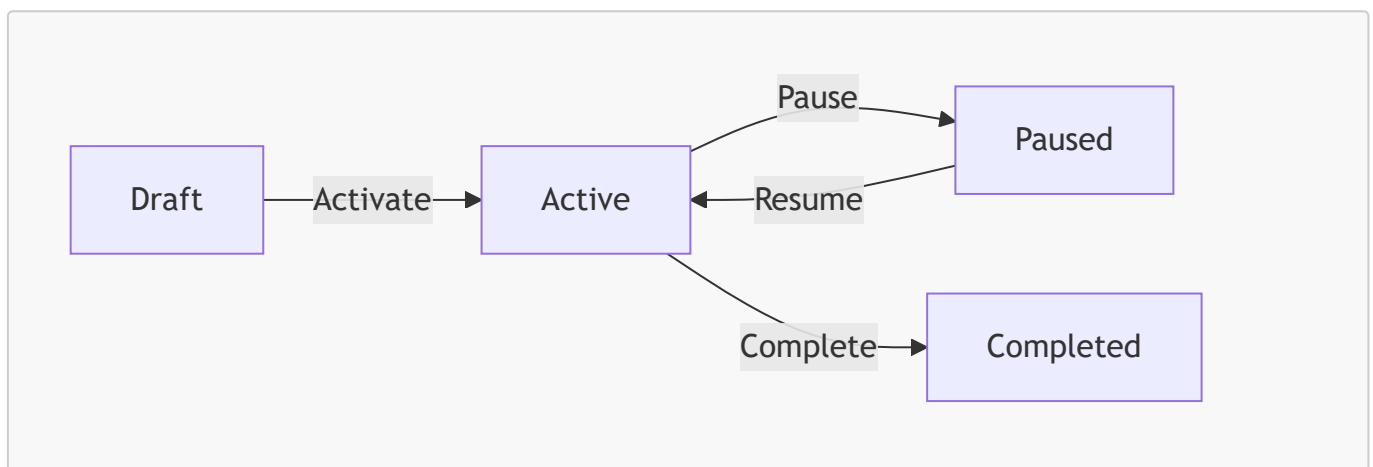
Editor/Advertiser → "Request Role" button → Submit request with reason
→ Admin reviews in Admin Console → Approve/Reject
→ Role updated immediately if approved

6. Application Flow

6.1 User Journey (End-to-End)



6.2 Campaign Lifecycle

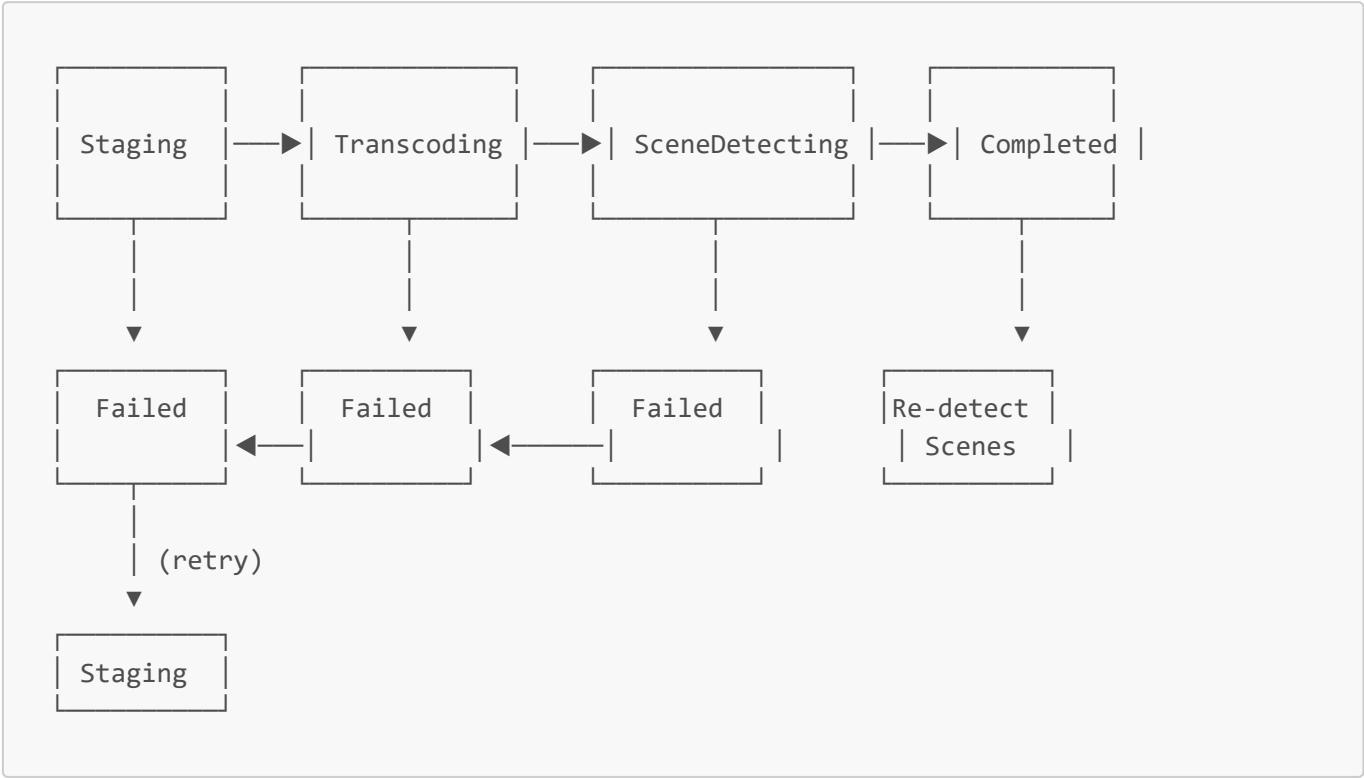


6.3 URL Routing Structure

/	→ Landing page (no campaign selected)
/c/:campaignId	→ Campaign Dashboard
/c/:campaignId/assets	→ Asset Management
/c/:campaignId/content	→ Content Ingestion
/c/:campaignId/placements	→ QA Workbench & Placements
/c/:campaignId/renderers	→ GPU Compositing & Renders
/c/:campaignId/reports	→ Campaign Reports
/admin	→ Admin Console
/telemetry	→ Telemetry (Logs & Alarms)
/analytics	→ Platform Analytics

7. Content Pipeline

7.1 Pipeline Stages



7.2 Stage Transitions

From	To	Trigger
Staging	Transcoding	Automatic after upload
Staging	Failed	Upload/validation error
Transcoding	SceneDetecting	Automatic after transcode
Transcoding	Failed	Transcode error
SceneDetecting	Completed	Automatic after detection
SceneDetecting	Failed	Detection error

From	To	Trigger
Completed	SceneDetecting	Manual "Re-detect Scenes"
Failed	Staging	Manual "Reset Pipeline"
Any	Failed	Manual "Mark Failed"

7.3 Pipeline Timestamps

Each content item tracks precise timestamps for every pipeline stage:

- `StagingCompletedAt`
- `TranscodingStartedAt` / `TranscodingCompletedAt`
- `SceneDetectingStartedAt` / `SceneDetectingCompletedAt`
- `LastErrorMessage` / `LastErrorAt`

7.4 Upload System

File Size	Method	Features
≤ 100 MB	Direct XHR Upload	Progress bar, real-time percentage
> 100 MB	Chunked Upload	25 MB chunks, 3 concurrent, resume support

8. API Surface

8.1 API Endpoints Summary

Controller	Base Path	Key Endpoints
Auth	<code>/api/auth</code>	<code>POST /login</code> , <code>POST /refresh</code> , <code>POST /forgot-password</code> , <code>POST /change-password</code>
Users	<code>/api/users</code>	<code>GET /</code> , <code>POST /</code> , <code>PUT /{id}</code> , <code>DELETE /{id}</code>
User Profile	<code>/api/user</code>	<code>GET /profile</code> , <code>PUT /notifications</code> , <code>POST /request-role</code>
Admin Role Requests	<code>/api/admin/role-requests</code>	<code>GET /</code> , <code>POST /{id}/approve</code> , <code>POST /{id}/reject</code>
Content	<code>/api/content</code>	<code>GET /</code> , <code>POST /upload</code> , <code>DELETE /{id}</code> , <code>POST /{id}/transition</code> , <code>POST /{id}/retranscode</code> , <code>POST /{id}/redetect-scenes</code> , <code>POST /{id}/mark-failed</code> , <code>POST /{id}/reset</code> , <code>GET /{id}/scenes</code>
Scenes	<code>/api/scenes</code>	<code>GET /{id}/surfaces</code> , <code>POST /update</code> , <code>POST /ai-suggest-assets</code>
Surfaces	<code>/api/surfaces</code>	<code>POST /{id}/approve</code>
Campaigns	<code>/api/campaigns</code>	<code>GET /</code> , <code>POST /</code> , <code>PUT /{id}</code> , <code>DELETE /{id}</code>

Controller	Base Path	Key Endpoints
Assets	/api/assets	GET /, POST /, POST /upload, PUT /{id}, PUT /{id}/upload, DELETE /{id}, PUT /{id}/campaign/{campaignId}, PUT /{id}/unassociate
Renderers	/api/renderers	GET /, POST /
Compositing	/api/compositing	POST /preview
Approvals	/api/approvals	GET /, POST /
Alarms	/api/alarms	GET /, POST /trigger, POST /{id}/clear
Logs	/api/logs	GET /, POST /
Notifications	/api/notifications	GET /, POST /
Stats	/api/stats	GET /summary
Usage	/api/usage	GET /, POST /archive
Brand Safety	/api/admin/brand-safety	GET /, POST /, POST /{id}/toggle
Admin Settings	/api/admin/settings	GET /, PUT /, POST /test-email
Attention	/api/attention	GET /, POST /

8.2 Pagination

All list endpoints support pagination with a consistent response shape:

```
{
  "items": [...],
  "totalCount": 150,
  "page": 1,
  "pageSize": 25,
  "totalPages": 6,
  "hasPreviousPage": false,
  "hasNextPage": true
}
```

Query parameters: ?page=1&pageSize=25&severity=Critical&search=keyword

9. Security Architecture

9.1 Authentication Flow

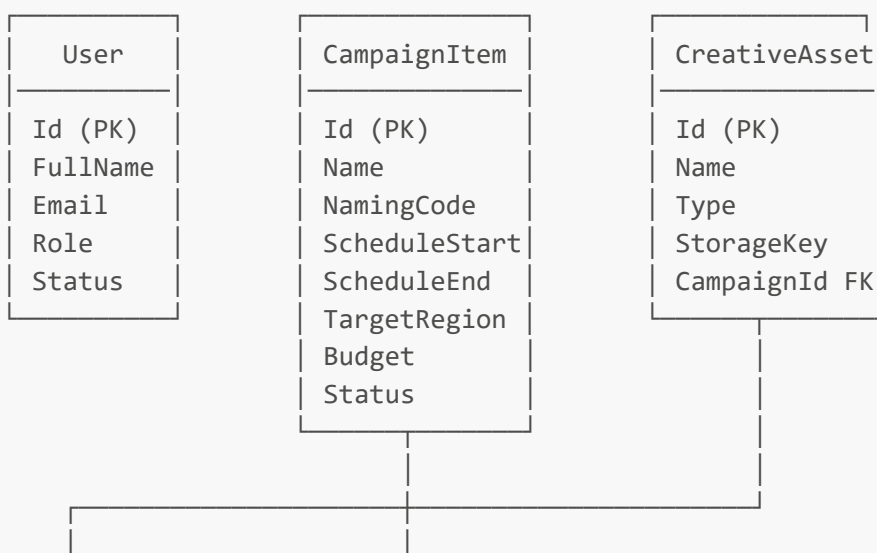
```
User → Login Form → POST /api/auth/login
  → Server validates email + BCrypt password hash
  → Server returns JWT token (8hr expiry) + user session
  → Client stores token in localStorage
  → All subsequent requests include: Authorization: Bearer <token>
  → Silent token refresh every 30 minutes (2hr refresh window)
  → Idle timeout: 28 min inactivity → 60s countdown → auto-logout
```

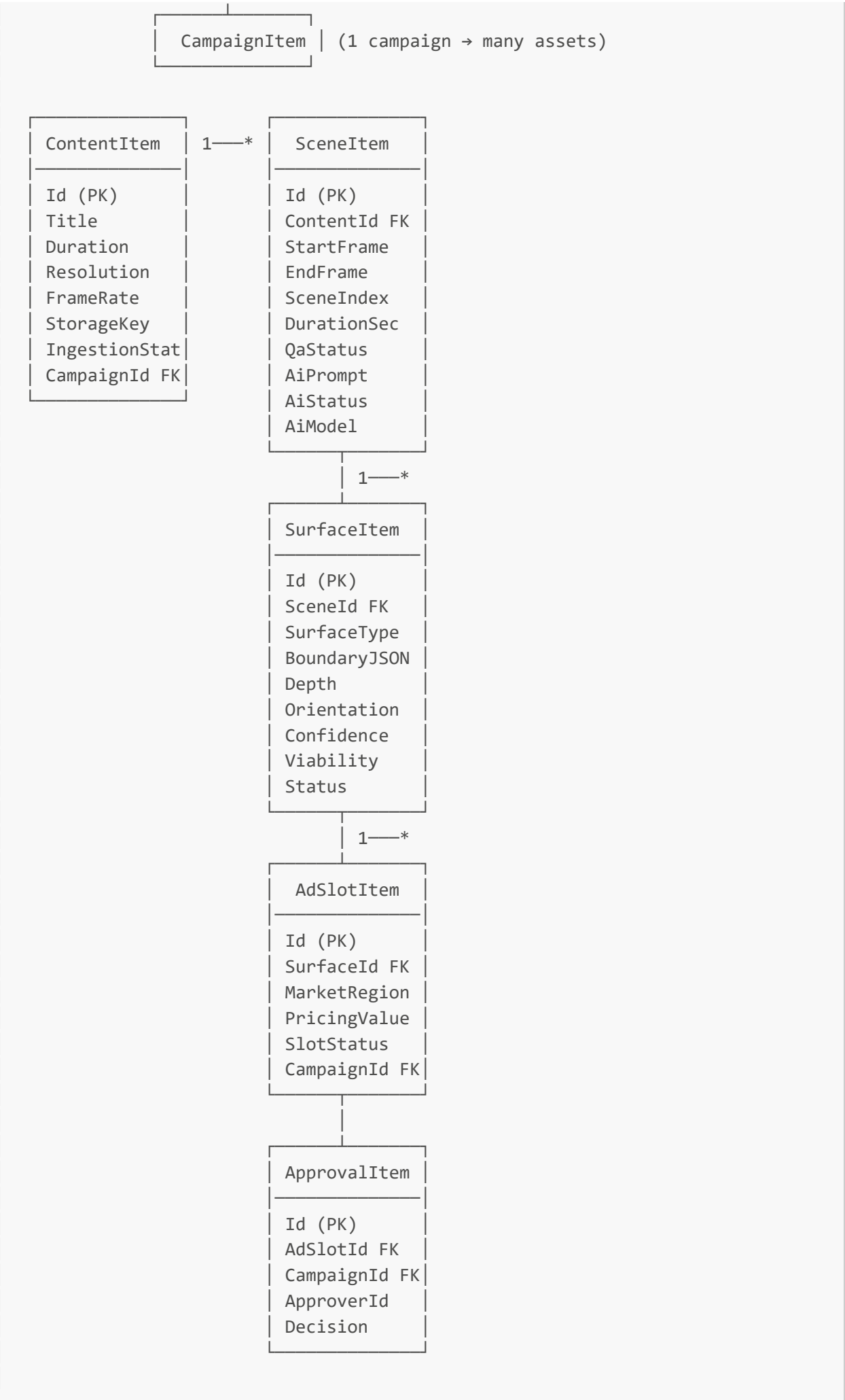
9.2 Security Measures

Measure	Implementation
Password Hashing	BCrypt (BCrypt.Net-Next)
JWT Signing	HMAC-SHA256 with configurable secret
Token Expiry	8 hours (configurable via <code>Jwt:ExpiryHours</code>)
Token Refresh	Silent refresh within 2-hour window (<code>Jwt:RefreshWindowHours</code>)
Idle Timeout	28 minutes → 60s countdown → forced logout
CORS	Whitelist: <code>localhost:3000</code> , <code>*.run.app</code>
Exception Handling	Production middleware catches unhandled errors, returns generic messages
Error Interception	Frontend intercepts 500-range and network errors with friendly messages
Authorization	Role-based access control on both frontend (URL guard) and backend (<code>[Authorize]</code> attributes)

10. Database Schema

10.1 Entity Relationship Diagram







10.2 Operational Entities

Entity	Purpose
EventLog	System event logging (auth, ingestion, AI, GPU, errors)
AlarmItem	Active/resolved alarms with severity (Minor/Major/Critical)
UsageRecord	Per-user API request tracking
NotificationItem	Email/SMS notification queue
PlatformSetting	Runtime-configurable settings (DB-backed, appsettings.json fallback)
RoleRequest	Role elevation requests with approval workflow
BrandSafetyRule	Permanent exclusion categories (add-only, toggle active/inactive)
PasswordResetToken	Time-limited password reset tokens

10.3 Seed Data Summary

When the application starts in Development mode, the following seed data is populated:

Entity	Count	Examples
Users	3	Admin, Editor, Advertiser
ContentItems	3	Soccer derby (1080p/50fps), drone survey (4K/60fps), living room test (1080p/25fps)
SceneItems	5	3 scenes for derby, 2 scenes for drone footage
SurfaceItems	5	LED board, face (excluded), grass mat, Coca-Cola sign (excluded), highway gantry
CampaignItems	3	Coca-Cola SADC (Active), Nike AirMax (Active), Samsung Neo-QLED (Draft)
CreativeAssets	3	Coke banner, Nike swoosh, Samsung video overlay
RenderItems	1	Finished ProRes render (42.5s processing)

Entity	Count	Examples
EventLogs	4	Auth success, FFmpeg metadata, brand-safety exclusion, GPU export
AlarmItems	2	Minor SMTP delay (cleared), Critical GPU VRAM timeout (active)

Appendix A: Quick Start

Prerequisites

- Node.js 18+
- .NET 8 SDK
- PostgreSQL 16+ (local or remote)
- Gemini API key (for AI features; optional for basic mode)

Startup Commands

```
# 1. Install frontend dependencies
npm install

# 2. Set Gemini API key (optional)
# Create .env.local with: GEMINI_API_KEY=your_key_here

# 3. Ensure PostgreSQL is running with database 'afrobotics_bit'
# Connection:
Host=localhost;Database=afrobotics_bit;Username=postgres;Password=Password@1

# 4. Start backend (applies migrations + seeds on first run)
dotnet run --project dotnet-api/Afrobotics.Bit.Api.csproj

# 5. Start frontend (separate terminal)
npx vite --host 0.0.0.0 --port 3000

# 6. Open browser
# http://localhost:3000
```

Login Credentials

Role	Email	Password
Admin	admin@afrobotics.co.za	admin123
Editor	loverboy.sfiso@gmail.com	editor123
Advertiser	advertiser@afrobotics.co.za	advertiser123

Appendix B: File Structure

BIT-APP/

└─ index.html	# SPA entry point
└─ package.json	# Frontend dependencies & scripts
└─ tsconfig.json	# TypeScript configuration
└─ vite.config.ts	# Vite build config + API proxy
└─ metadata.json	# AI Studio metadata
└─ README.md	# Original AI Studio README
└─ docs/	
└─ DESIGN_DOCUMENT.md	# Full system design specification
└─ APPLICATION_PRESENTATION.md	# ← This file
└─ src/	# React Frontend
└─ main.tsx	# App entry (BrowserRouter)
└─ App.tsx	# Root component (auth, routing, state)
└─ index.css	# Tailwind imports + dark mode overrides
└─ types.ts	# TypeScript interfaces & constants
└─ apiClient.ts	# HTTP client (JWT, pagination, login)
└─ document.ts	# Implementation plan document
└─ components/	
└─ AdminConsoleTab.tsx	# User management + admin panels
└─ AnalyticsTab.tsx	# Platform analytics dashboard
└─ AttentionBell.tsx	# Notification bell
└─ BrandSafetyPanel.tsx	# Brand-safety exclusion list
└─ CampaignDashboard.tsx	# Campaign-level dashboard
└─ CampaignSelector.tsx	# Campaign dropdown picker
└─ CampaignSidebar.tsx	# URL-driven sidebar navigation
└─ CampaignsTab.tsx	# Asset management & campaign view
└─ ComposerTab.tsx	# GPU render queue & stitching
└─ EditorTab.tsx	# QA workbench & placement editor
└─ FilterableSelect.tsx	# Searchable dropdown component
└─ IngestionTab.tsx	# Video upload & pipeline control
└─ NotFoundPage.tsx	# 404 page
└─ NotificationPreferencesPanel.tsx	
└─ Pagination.tsx	# Reusable pagination component
└─ PipelineProgress.tsx	# 4-stage pipeline indicator
└─ RoleRequestsPanel.tsx	# Admin role request management
└─ SettingsPanel.tsx	# Platform settings editor
└─ TelemetryTab.tsx	# Logs & alarms viewer
└─ hooks/	
└─ useChunkedUpload.ts	# Chunked upload hook
└─ useIdleTimer.ts	# Session idle timeout hook
└─ usePaginatedData.ts	# Paginated API data hook
└─ dotnet-api/	# .NET 8 Backend
└─ Afrobotics.Bit.Api.csproj	# Project file with NuGet references
└─ Program.cs	# App startup, DI, middleware, Hangfire
└─ appsettings.json	# Configuration (DB, JWT, SMTP, engines)
└─ HangfireDashboardAuthFilter.cs	# Hangfire dashboard auth
└─ Controllers/	# 20 API controllers
└─ Models/Models.cs	# All entity classes
└─ Data/	
└─ PostgresDbContext.cs	# EF Core DbContext

```
| | └─ DbSeeder.cs           # Development seed data
| | └─ DTOs/                 # Data transfer objects
| | └─ Middleware/           # Exception & usage tracking
| | └─ Migrations/           # EF Core migrations
| | └─ Repositories/         # Generic + specialized repos
| | └─ Services/             # 27 service files
|
| └─ dotnet-api.Tests/       # xUnit test project
|   └─ Afrobotics.Bit.Tests.csproj
|   └─ AuthServiceTests.cs
|   └─ CampaignServiceTests.cs
|   └─ SurfaceServiceTests.cs
|
| └─ .github/
|   └─ agents/
|     └─ plan-executor.agent.md # Plan execution agent definition
```

End of Document — Afrobotics BIT Application Presentation v1.0